MATERI PEMBELAJARAN KUIS
GRAFIKA KOMPUTER

Mencakup: Transformasi 2D/3D • Lighting • Shader • WebGL Pipeline
Texture Mapping • Procedural Modeling

1. Matriks Transformasi 2D

Catatan Kuis: 4 soal transformasi: translasi, rotasi, rotasi+translasi, membuat matriks dari perubahan titik.

1.1 Konsep Dasar Transformasi Affine

Transformasi affine menjaga kesejajaran garis. Bentuk umum:

```
x1 = a*x + b*y + e
y1 = c*x + d*y + f

Notasi: T(x,y) = ( a*x + b*y + e,  c*x + d*y + f )
```

1.2 Translasi

Memindahkan titik sejauh (e, f).

```
x1 = x + e
y1 = y + f

Matriks homogen 3x3:
| 1  0  e | | x |   | x + e |
| 0  1  f | | y | = | y + f |
| 0  0  1 | | 1 |   |   1   |
```

1.3 Rotasi

Memutar titik sebesar r radian terhadap origin (0,0). Positif = berlawanan jarum jam.

```
x1 = cos(r) * x - sin(r) * y
y1 = sin(r) * x + cos(r) * y

Matriks homogen:
| cos(r)  -sin(r)  0 |
| sin(r)   cos(r)  0 |
|   0        0     1 |
```

Catatan Kuis: Rotasi terhadap titik (p,q): translate(-p,-q) lalu rotate(r) lalu translate(p,q).

1.4 Scaling

```
x1 = a * x
y1 = d * y

Matriks:
| a  0  0 |
| 0  d  0 |
| 0  0  1 |
```

1.5 Kombinasi Transformasi

Kombinasi = perkalian matriks. Urutan PENTING — matriks paling kanan diterapkan PERTAMA.

```
# Rotasi LALU translasi:
M = T(e,f) * R(r)     ← R diterapkan dulu ke titik
```

```
# translate(4,2) lalu translate(-1,5) = translate(3,7)
| 1 0 4 |   | 1 0 -1 |   | 1 0 3 |
| 0 1 2 | * | 0 1  5 | = | 0 1 7 |
| 0 0 1 |   | 0 0  1 |   | 0 0 1 |
```

1.6 Contoh Soal Transformasi

Soal 1 — Matriks Translasi

Titik P(3,4) ditranslasi tx=2, ty=-1.

```
Matriks T(2,-1):     Hasil:
| 1  0  2 |         | 1  0  2 | | 3 |   | 5 |
| 0  1 -1 |         | 0  1 -1 | | 4 | = | 3 |
| 0  0  1 |         | 0  0  1 | | 1 |   | 1 |

P baru = (5, 3)
```

Soal 2 — Matriks Rotasi

Titik P(1,0) dirotasi 90°. cos(90)=0, sin(90)=1.

```
x1 = cos(90)*1 - sin(90)*0 = 0
y1 = sin(90)*1 + cos(90)*0 = 1

P baru = (0, 1)

Matriks R(90°):
|  0  -1  0 |
|  1   0  0 |
|  0   0  1 |
```

Soal 3 — Rotasi lalu Translasi

Titik P(1,0), rotasi 90° lalu translasi (2,3).

```
Langkah 1 — Rotasi 90°:  P_rot = (0, 1)
Langkah 2 — Translasi:   P_final = (0+2, 1+3) = (2, 4)

Matriks gabungan M = T(2,3) * R(90°):
| 1 0 2 |   |  0 -1 0 |   |  0 -1 2 |
| 0 1 3 | * |  1  0 0 | = |  1  0 3 |
| 0 0 1 |   |  0  0 1 |   |  0  0 1 |
```

Soal 4 — Membuat Matriks dari Perubahan Titik

A(0,0)→A'(3,2) dan B(1,0)→B'(4,2). Cari matriks transformasi.

```
Kedua titik bergeser (+3, +2) → ini translasi T(3,2)

Matriks:
| 1 0 3 |
| 0 1 2 |
| 0 0 1 |

Verifikasi:
```

```
A(0,0): x=0+3=3, y=0+2=2 → (3,2) ✓
B(1,0): x=1+3=4, y=0+2=2 → (4,2) ✓
```

2. Lighting (Pencahayaan)

Catatan Kuis: Menjelaskan rumus lighting tiap bagian dan efeknya.

2.1 Mengapa Shading Penting

Tanpa shading, objek 3D terlihat datar. Shading mensimulasikan interaksi sumber cahaya dengan permukaan → memberikan kesan tiga dimensi.

Faktor yang mempengaruhi shading:

- Sumber cahaya: tipe, posisi, warna/intensitas
- Material objek: warna diffuse, specular, ambient, emission
- Posisi viewer
- Orientasi permukaan (normal vector)

2.2 Persamaan Lighting Phong — Lengkap

Komponen merah (r) warna vertex:

```
r = me_r + ga_r*ma_r + I_0,r + I_1,r + ...

Kontribusi satu sumber cahaya:
I_r = la_r*ma_r  +  f * ( ld_r*md_r*(L·N)  +  ls_r*ms_r*max(0,V·R)^mh )

Persamaan sama untuk komponen g dan b.
```

2.3 Penjelasan Setiap Komponen

a. Emisi (me)

Warna yang dipancarkan objek sendiri. Langsung ditambahkan. Biasanya (0,0,0).

```
r = me_r + ...
```

b. Global Ambient (ga × ma)

Cahaya ambient lingkungan × warna ambient material. Menerangi semua sisi secara merata.

```
Kontribusi = ga_r * ma_r
```

c. Ambient Cahaya (la × ma)

Komponen ambient dari sumber cahaya spesifik × warna ambient material.

```
= la_r * ma_r
```

d. Diffuse (ld × md × (L·N))

Cahaya yang tersebar merata. Bergantung pada sudut antara L (arah cahaya) dan N (normal).

- ld = diffuse color sumber cahaya
- md = diffuse color material = WARNA OBJEK
- L = vektor dari permukaan ke sumber cahaya (unit)
- N = normal vector permukaan (unit)

- L·N = cos(θ) — makin tegak lurus, makin terang

```
Kontribusi = f * ld_r * md_r * (L·N)

# f=0: permukaan tidak menghadap cahaya
# f=1: permukaan menghadap cahaya
```

e. Specular (ls × ms × max(0, V·R)^mh)

Highlight/kilatan pada permukaan mengkilap.

- ls = specular color sumber cahaya
- ms = specular color material
- V = vektor menuju viewer (unit)
- R = arah reflected ray = 2*(N·L)*N − L
- mh = shininess (0-128): makin tinggi ⟶ highlight makin kecil dan tajam

```
Kontribusi = f * ls_r * ms_r * max(0, V·R)^mh

// GLSL:
R = -reflect(L, N);           // R = arah pantulan
V = normalize(-eyeCoords.xyz); // V = arah ke viewer (perspective)
// atau V = vec3(0,0,1) untuk orthographic projection
```

2.4 Normal Vector

Vektor tegak lurus permukaan, panjang = 1. Wajib dalam koordinat eye sebelum digunakan:

```
// Di vertex shader:
vec3 N = normalize( normalMatrix * normal );
// normalMatrix = transpose(inverse(modelview))
```

Flat vs Smooth shading: berbeda pada nilai normal vector di setiap vertex.

2.5 Per-Vertex vs Per-Pixel Lighting

- Per-vertex: perhitungan di vertex shader — lebih cepat, kurang akurat
- Per-pixel (Phong shading): perhitungan di fragment shader, normal diinterpolasi — lebih akurat

3. Vertex Shader vs Fragment Shader

Catatan Kuis: Diberikan potongan kode GLSL — tentukan fragment shader atau vertex shader.

3.1 Perbedaan Utama

Vertex shader: dieksekusi sekali per VERTEX.

Fragment shader: dieksekusi sekali per PIXEL (fragment).

3.2 Ciri-Ciri Vertex Shader

- Ada keyword "attribute" (data per-vertex dari JavaScript)
- Wajib ada penugasan ke gl_Position
- Mendeklarasikan dan mengisi "varying" (dikirim ke fragment shader)
- Beroperasi pada koordinat 3D/4D
- TIDAK ada "precision mediump float;" di awal

```
// CONTOH VERTEX SHADER:
attribute vec4 aPosition;      // ← ada attribute
attribute vec4 aColor;
varying vec4 vColor;            // ← varying di-assign
uniform mat4 uModelviewMatrix;

void main() {
    gl_Position = uModelviewMatrix * aPosition;  // ← gl_Position wajib
    vColor = aColor;             // ← mengisi varying
}
```

3.3 Ciri-Ciri Fragment Shader

- "precision mediump float;" di baris paling atas (GLSL ES 1.0)
- Wajib ada penugasan ke gl_FragColor
- Varying hanya DIBACA (sudah diinterpolasi oleh rasterizer)
- Tidak ada "attribute"
- Bisa menggunakan texture2D() untuk sampling texture

```
// CONTOH FRAGMENT SHADER:
precision mediump float;        // ← penanda fragment shader
varying vec4 vColor;             // ← hanya dibaca
uniform sampler2D uSampler;

void main() {
    gl_FragColor = vColor;        // ← gl_FragColor wajib
}
```

3.4 Contoh Soal — Identifikasi Shader

Contoh A:

```
precision mediump float;
varying vec4 vColor;
uniform vec4 uAmbient;
void main() {
```

```
    gl_FragColor = uAmbient + vColor;
}
```

Jawaban: FRAGMENT SHADER — ada "precision mediump float" dan "gl_FragColor".

Contoh B:

```
attribute vec4 aPosition;
attribute vec4 aColor;
varying vec4 vColor;
void main() {
    gl_Position = aPosition;
    vColor = aColor;
}
```

Jawaban: VERTEX SHADER — ada "attribute" dan "gl_Position".

Contoh C (Lighting):

```
attribute vec3 a_coords;
attribute vec3 a_normal;
uniform mat4 modelview;
uniform mat3 normalMatrix;
varying vec3 v_normal;
varying vec4 v_eyeCoords;
void main() {
    vec4 eyeCoords = modelview * vec4(a_coords, 1.0);
    gl_Position = projection * eyeCoords;
    v_normal = normalMatrix * a_normal;
    v_eyeCoords = eyeCoords;
}
```

Jawaban: VERTEX SHADER — ada "attribute", "gl_Position", varying di-assign.

Contoh D (Texture + Lighting Fragment):

```
precision mediump float;
varying vec4 v_color;
varying vec2 v_texCoords;
uniform sampler2D u_sampler;
void main() {
    vec4 texColor = texture2D(u_sampler, v_texCoords);
    gl_FragColor = v_color * texColor;
}
```

Jawaban: FRAGMENT SHADER — "precision", "gl_FragColor", dan texture2D().

4. Triangle Strip — Menghitung Jumlah Vertex

Catatan Kuis: 12 segitiga dari triangle strip butuh berapa vertex?

4.1 Jenis Primitive WebGL

- gl.TRIANGLES — 3 vertex per segitiga, tidak ada sharing
- gl.TRIANGLE_STRIP — berbagi 2 vertex dengan segitiga sebelumnya
- gl.TRIANGLE_FAN — semua segitiga berbagi 1 vertex pusat

4.2 Rumus Jumlah Vertex

```
gl.TRIANGLES:       n segitiga → 3n vertex
gl.TRIANGLE_STRIP:  n segitiga → n+2 vertex
gl.TRIANGLE_FAN:    n segitiga → n+2 vertex

12 segitiga:
  TRIANGLES:      12 * 3 = 36 vertex
  TRIANGLE_STRIP: 12 + 2 = 14 vertex  ← jawaban kuis
  TRIANGLE_FAN:   12 + 2 = 14 vertex
```

4.3 Penjelasan TRIANGLE_STRIP

Vertex ke-1,2,3 membentuk segitiga pertama. Setiap vertex baru + 2 vertex sebelumnya = segitiga baru.

```
V1-V2-V3 = segitiga 1
V2-V3-V4 = segitiga 2
V3-V4-V5 = segitiga 3
...
Vn-V(n+1)-V(n+2) = segitiga n

→ 5 vertex untuk 3 segitiga  (3+2=5)
→ 14 vertex untuk 12 segitiga (12+2=14)

gl.drawArrays(gl.TRIANGLE_STRIP, 0, 14);  // 12 segitiga
```

5. Programmable Pipeline

Catatan Kuis: Menjelaskan programmable pipeline.

5.1 Fixed vs Programmable Pipeline

Fixed-function pipeline (OpenGL lama): semua tahapan sudah ditentukan, programmer tidak bisa mengubah cara rendering. Programmable pipeline (WebGL/modern OpenGL): programmer menulis shader untuk menggantikan beberapa tahapan.

WebGL menggunakan programmable pipeline secara WAJIB (mandatory).

5.2 Empat Tahapan Pipeline

1. Vertex Processing (PROGRAMMABLE) — Vertex Shader — sekali per vertex
2. Clipping & Primitive Assembly (FIXED) — memotong di luar view volume
3. Rasterization (FIXED) — konversi geometri ke fragment, interpolasi varying
4. Fragment Processing (PROGRAMMABLE) — Fragment Shader — sekali per pixel

5.3 Alur Data

```
JavaScript (CPU)
  │  attribute data (per-vertex: koordinat, warna, normal)
  │  uniform data (sama untuk semua: matriks transformasi)
  ↓
VERTEX SHADER (GPU)
  │  → gl_Position (clip coordinates) — WAJIB
  │  → varying variables (dikirim ke fragment shader)
  ↓
Clipping + Rasterization [fixed]
  │  → interpolasi varying
  ↓
FRAGMENT SHADER (GPU)
  │  ← varying (sudah diinterpolasi)
  │  ← uniform
  │  → gl_FragColor — WAJIB
  ↓
Frame Buffer (layar)
```

5.4 Tiga Jenis Variabel GLSL

attribute: Nilai berbeda per vertex. Hanya di vertex shader. Dari JS via VBO.

uniform: Nilai sama untuk seluruh primitive. Di vertex dan/atau fragment shader. Dari JS langsung.

varying: Di-assign di vertex shader, diinterpolasi rasterizer, dibaca di fragment shader.

5.5 Mengapa Programmable Pipeline?

- Kontrol penuh atas efek visual: lighting, shadow, texture, post-processing
- Shader berjalan di GPU → sangat cepat (paralel)
- Three.js menggunakan program berbeda untuk setiap tipe Material

- Dapat membuat beberapa program shader dan switch antar-program

6. Texture Mapping (Konsep)

Catatan Kuis: Hanya konsep, tidak ada perhitungan spesifik.

6.1 Definisi

Texture mapping menambahkan variasi warna pixel-by-pixel pada permukaan objek, seperti melukis gambar pada permukaan 3D. Membuat objek terlihat lebih detail dan realistis.

6.2 Koordinat Tekstur (s, t)

- s = koordinat horizontal: 0.0 (kiri) hingga 1.0 (kanan)
- t = koordinat vertikal: 0.0 (bawah) hingga 1.0 (atas)

Setiap vertex memiliki texture coordinate (s,t). Titik dalam primitif diinterpolasi.

6.3 Alur di WebGL

5. Load image texture ke GPU sebagai texture object
6. Vertex shader terima a_texCoords sebagai attribute
7. Vertex shader teruskan ke fragment shader via v_texCoords (varying)
8. Fragment shader ambil warna dengan texture2D(sampler, coords)

```
// VERTEX SHADER:
attribute vec2 a_texCoords;
varying vec2 v_texCoords;
void main() { ...; v_texCoords = a_texCoords; }

// FRAGMENT SHADER:
precision mediump float;
varying vec2 v_texCoords;
uniform sampler2D u_sampler;
void main() {
    vec4 texColor = texture2D(u_sampler, v_texCoords);
    gl_FragColor = texColor;
}
```

6.4 Detail Penting

- gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, 1) — flip Y agar orientasi texture benar
- sampler2D = uniform type untuk texture 2D
- texture2D(sampler, coord) — mengambil warna dari posisi (s,t)

7. Presentasi Kelompok: Procedural Modeling

Catatan Kuis: Materi presentasi kelompok — folder 9.PM.

7.1 Definisi

Procedural modeling adalah pembuatan geometri 3D menggunakan program/algoritma, bukan pemodelan manual. Program menghasilkan vertex dan face secara otomatis.

Keunggulan:

- Variasi tanpa manual re-work (ubah seed → geometri berbeda)
- Representasi kompak (100 baris kode = terrain kompleks)
- Art style khas: terrain, kota, vegetasi prosedural

7.2 Tiga Tipe yang Dipresentasikan

9. Procedural Noise (Perlin Noise / fBm)
10. Rule-Based Modeling (L-System)
11. Signed Distance Fields (SDF)

7.3 Procedural Noise & Perlin Noise

Smoothstep Function

Fungsi penting dalam shader programming:

```
S(λ) = 3λ² - 2λ³           (smoothstep)
S(0)=0, S(1)=1, S'(0)=0, S'(1)=0

S₂(λ) = 6λ⁵ - 15λ⁴ + 10λ³  (smootherstep, Ken Perlin 2002)
Tambahan: S''(0)=0, S''(1)=0
```

Perlin Noise

Interpolasi Hermite + nilai acak dari hash function di corner tile grid:

```
float noise( in vec2 x ) {
    vec2 p = floor(x);              // koordinat tile
    vec2 w = fract(x);              // posisi dalam tile
    vec2 u = smoothstep(0.0, 1.0, w); // S(w)
    float a = hash(p+vec2(0,0));
    float b = hash(p+vec2(1,0));
    float c = hash(p+vec2(0,1));
    float d = hash(p+vec2(1,1));
    return -1.0+2.0*(a + (b-a)*u.x + (c-a)*u.y + (a-b-c+d)*u.x*u.y);
}
```

fBm (Fractal Brownian Motion) / Turbulence

Jumlahkan noise di banyak skala untuk detail multi-level:

```
y = noise(p) + (1/2)*noise(2*R*p) + (1/4)*noise(4*R²*p) + ...

Setiap octave: frekuensi ×2, amplitudo ×0.5
```

```
R = rotation matrix (mencegah artifact sejajar sumbu)

float fbm_9( in vec2 x ) {
    mat2 R = mat2( 0.80, 0.60, -0.60, 0.80 );
    float f = 1.9;   float s = 0.55;
    float a = 0.0;   float b = 0.5;
    for( int i=0; i<9; i++ ) {
        a += b * noise(x);
        b *= s;   x = f*R*x;
    }
    return a;
}
```

Aplikasi: No Man's Sky generate planet menggunakan fBm.

7.4 L-System (Rule-Based Modeling)

Konsep

Grammar formal yang menulis ulang string simbol berdasarkan production rules. String hasil diinterpretasikan sebagai instruksi turtle graphics.

Fractal Binary Tree

```
Rules: 0 → 1[0]0    1 → 11

Ekspansi:
  Step 0: 0
  Step 1: 1[0]0
  Step 2: 11[1[0]0]1[0]0

Interpretasi:
  0 = gambar garis, pop stack
  1 = gambar garis, lanjut
  [ = push posisi & arah, belok kiri 45°
  ] = pop posisi & arah, belok kanan 45°
```

Aplikasi

- SpeedTree — vegetasi untuk AAA games
- CityEngine (Esri) — generasi kota prosedural
- Fraktal: Sierpinski triangle, Koch snowflake

7.5 Signed Distance Fields (SDF)

Definisi

SDF adalah fungsi implisit di mana nilai = signed distance ke permukaan terdekat:

```
f(x,y,z) = distance * sign
  sign = +1 jika di luar permukaan
  sign = -1 jika di dalam permukaan

Permukaan = {(x,y,z) | f(x,y,z) = 0}
```

```
// SDF sphere radius r:
f(x,y,z) = sqrt(x²+y²+z²) - r
```

Operasi Boolean

```
Union (gabung):          min(f1, f2)
Intersection (irisan):   max(f1, f2)
Subtraction (kurangi):   max(f1, -f2)
Smooth Union:            smin(f1, f2, k)  ← blend halus
```

Perbandingan Tiga Teknik

```
Procedural Noise:
  + Cepat, smooth, variasi tak terbatas
  - Sulit kontrol tepat
  Untuk: terrain, awan, air

L-System:
  + Biologically plausible, self-similar
  - Susah tuning rules
  Untuk: pohon, kota, fraktal

SDF:
  + Boolean bersih, smooth blend, ray marchable
  - Tidak ada explicit mesh
  Untuk: bentuk organis, real-time sculpting
```

Ringkasan Kilat untuk Kuis

A. Matriks Transformasi — Hafalan

```
Translasi T(e,f):      Rotasi R(r):          Scaling S(a,d):
| 1 0 e |              | cos -sin 0 |         | a 0 0 |
| 0 1 f |              | sin  cos 0 |         | 0 d 0 |
| 0 0 1 |              |  0    0  1 |         | 0 0 1 |

Urutan: M = T * R  →  R diterapkan DULU ke titik
```

B. Persamaan Lighting — Hafalan

```
I_r = la_r*ma_r + f*( ld_r*md_r*(L·N) + ls_r*ms_r*max(0,V·R)^mh )

la=ambient light  ma=ambient material
ld=diffuse light  md=diffuse material (warna objek)
ls=specular light ms=specular material
mh=shininess      L=arah cahaya  N=normal
V=arah viewer     R=reflected ray
f=1 menghadap cahaya, f=0 tidak
L·N = cos(θ) sudut antara cahaya dan normal
```

C. Identifikasi Shader — Kunci Cepat

```
VERTEX SHADER                FRAGMENT SHADER
  attribute ...                precision mediump float;
  gl_Position = ...;           gl_FragColor = ...;
  varying var = assign         texture2D()
  TIDAK ada precision          varying hanya dibaca
                               TIDAK ada attribute
```

D. Triangle Strip — Rumus

```
TRIANGLE_STRIP / FAN:  n segitiga → n+2 vertex
TRIANGLES biasa:       n segitiga → 3n vertex

12 segitiga STRIP = 12+2 = 14 vertex
```

E. Pipeline — Urutan

```
1. Vertex Shader    (programmable) ← attribute, uniform → gl_Position,
varying
2. Clipping & Assembly (fixed)
3. Rasterization    (fixed)        ← interpolasi varying
4. Fragment Shader  (programmable) ← varying, uniform → gl_FragColor
```

F. Procedural Modeling

```
1. Perlin Noise / fBm:
   S(λ) = 3λ²-2λ³  (smoothstep)
   fBm = Σ noise(2^i*p) / 2^i  (multi-octave)

2. L-System: grammar rules + turtle graphics
   [ = push, ] = pop, F = draw forward

3. SDF: f(p) = signed distance to surface
   Union=min, Intersect=max, Subtract=max(f1,-f2)
```